中图分类号：TK16（请核对分类号）

论 文 编 号：10006ZF2421124

北京航空航天大學

专业硕士学位论文

面向数据库 XML 类型的压缩与索引优化方法

学位论文初稿

作者姓名　周志强

专业名称　软件工程

指导教师　刘子鹏　副教授

培养学院　软件学院

Compression and Indexing Optimization for XML Types in Databases

Thesis Draft

A Dissertation Submitted for the Professional Degree of Master of Electronic Information

Candidate : **Zhou Zhiqiang**

Supervisor: **A.P. Liu Zipeng**

School of Software

Beihang University, Beijing, China

中图分类号：TK16（请核对分类号）

论 文 编 号：10006ZF2421124

硕 士 学 位 论 文

面向数据库 XML 类型的压缩与索引优化方法

学位论文初稿

作者姓名	周志强	申请学位级别	工学硕士
指导教师姓名	刘子鹏	职　　称	副教授
专业名称	软件工程	研究方向	数据库 XML 压缩与索引优化
学习时间自　　年　月　日起		至　　年　月　日止	
论文提交日期　年　月　日		论文答辩日期　年　月　日	
学位授予单位	北京航空航天大学	学位授予日期	年　月　日

关于学位论文的独创性声明

本人郑重声明：所呈交的论文是本人在指导教师指导下独立进行研究工作所取得的成果，论文中有关资料和数据是实事求是的。尽我所知，除文中已经加以标注和致谢外，本论文不包含其他人已经发表或撰写的研究成果，也不包含本人或他人为获得北京航空航天大学或其它教育机构的学位或学历证书而使用过的材料。与我一同工作的同志对研究所做的任何贡献均已在论文中作出了明确的说明。

若有不实之处，本人愿意承担相关法律责任。

学位论文作者签名：_____ 日期： 年 月 日

学位论文使用授权

本人完全同意北京航空航天大学有权使用本学位论文（包括但不限于其印刷版和电子版），使用方式包括但不限于：保留学位论文，按规定向国家有关部门（机构）送交学位论文，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索，采用影印、缩印或其他复制手段保存学位论文。

保密学位论文在解密后的使用授权同上。

学位论文作者签名：_____ 日期： 年 月 日

指导教师签名：_____ 日期： 年 月 日

摘　要

针对数据库中 XML 数据的存储与查询需求，本文研究一种面向查询的结构感知压缩与索引优化方法。该方法在解析 XML 时识别元素路径、文本值和节点结构，将路径及值映射为可复用标识，并组织为文档序内容流、路径辅助信息、Bloom Filter 元数据、长值内联区和结构流。查询与索引键提取复用编码阶段形成的标识，以减少重复解析和重复识别；对于 Bloom Filter 或哈希产生的候选，仍通过精确验证保证结果正确。本文在 PostgreSQL 扩展原型中实现相关数据表示、XPath 导向查询函数及 GIN 索引操作类，并通过正确性检查和实验观察评估其存储与查询表现。实验显示，在所测数据与配置下，压缩列值相较原生 XML 列值有所减少；与 text[] + GIN 工程基线的查询延迟接近。该结果体现了压缩、查询与索引之间的空间和处理代价权衡，不构成对其他可查询压缩系统或数据库平台的普遍性能结论。

关键词：XML，结构感知压缩，压缩域查询，路径索引，GIN 索引

Abstract

This thesis studies a structure-aware, query-oriented compression and indexing method for XML data stored in databases. During XML parsing, the method identifies element paths, text values, and node structure, maps paths and values to reusable identifiers, and organizes them into a document-order content stream, path metadata, Bloom Filter metadata, an inline area for long values, and a structural stream. Queries and index-key extraction reuse identifiers produced during encoding to reduce repeated parsing and recognition. Candidates produced by Bloom Filters or hashes are subject to exact verification to preserve result correctness. The method is implemented in a PostgreSQL extension prototype with a binary representation, XPath-oriented query functions, and a GIN operator class, and is evaluated through correctness checks and empirical measurements. Under the tested data and configuration, the compressed column values are smaller than native XML column values, while query latency is close to that of the text[] + GIN engineering baseline. These results illustrate trade-offs among compression, querying, and indexing; they do not establish general performance superiority over other queryable compressors or database platforms.

Keywords: XML, Structure-Aware Compression, Querying over Compressed Data, Path Index, GIN Index

目 录

第一章 绪论 …… 1

1.1 研究背景与意义 …… 1

1.1.1 XML 数据的特征与数据库应用背景 …… 1

1.1.2 XML 存储与查询的现实需求 …… 1

1.1.3 研究意义 …… 2

1.2 国内外研究现状 …… 2

1.2.1 XML 压缩方法研究现状 …… 2

1.2.2 可查询 XML 压缩研究现状 …… 3

1.2.3 XML 查询与索引研究现状 …… 3

1.2.4 研究现状小结 …… 4

1.3 研究目标与研究内容 …… 4

1.3.1 研究目标 …… 4

1.3.2 研究内容 …… 5

1.3.3 主要贡献 …… 5

1.3.4 研究范围与设计要求 …… 6

1.4 论文组织结构 …… 6

1.4.1 技术路线 …… 6

1.4.2 章节安排 …… 6

1.4.3 本章小结 …… 7

第二章 XML 压缩存储与查询的理论基础 …… 8

2.1 XML 数据模型及存储特征 …… 8

2.2 XML 压缩方法 …… 8

2.3 可查询 XML 压缩方法 …… 9

2.4 XML 路径查询与索引方法 …… 9

2.5 XML 内容查询与候选过滤 …… 10

2.6 字典编码、Varint 与 Bloom Filter …… 10

2.7 PostgreSQL 扩展机制与 GIN 索引 …… 11

2.8 本章小结 …… 11

第三章 可查询 XML 压缩方法及问题分析 …… 12

3.1 分析对象与比较维度 …… 12

3.2 编码阶段信息复用的分析 …… 13

3.3 查询候选生成与精确验证分析 …… 13

3.4 压缩、查询与索引代价的耦合关系 …… 14

3.5 本文研究问题、适用范围与设计要求 …… 15

3.6 本章小结 …… 15

第四章 面向查询的 XML 压缩表示方法 …… 16

4.1 方法总体思路与设计目标 …… 16

4.2 XML 结构与内容的编码表示 …… 18

4.2.1 贯穿本章的 XML 示例 …… 18

4.2.2 路径和值的标识化 …… 19

4.3 字典复用与变长整数编码 …… 22

4.4 面向查询的辅助数据组织 …… 23

4.5 压缩表示的解码与正确性约束 …… 23

4.6 本章小结 …… 24

第五章 基于压缩表示的查询处理方法 …… 25

5.1 查询模型与处理流程 …… 25

5.2 精确内容查询 …… 25

5.2.1 示例文档上的正例与负例 …… 26

5.3 路径存在性与值提取查询 …… 27

5.3.1 示例文档上的路径查询 …… 27

5.4 路径索引、Bloom 过滤与候选验证 …… 28

5.5 PostgreSQL 操作符及 GIN 索引实现 …… 28

5.6 本章小结 …… 29

第六章 实验评估与结果讨论 …… 30

6.1 研究问题与评价指标 …… 30

6.2 实验环境、数据集与对比方案 …… 30
6.3 编码正确性与存储开销分析 …… 31
6.4 查询性能与索引效果分析 …… 32
6.5 写入开销与消融实验 …… 34
6.6 结果分析与有效性威胁 …… 34
6.7 本章小结 …… 35
结　论 …… 36
参考文献 …… 38

插图清单

图 4.1 XML 编码与中间数据复用流程 …… 17

附表清单

表 3.1 代表性 XML 方法与本文的分析维度 …… 12

表 4.1 XML 示例的路径和值标识 …… 20

表 4.2 逻辑二进制布局及区段内容 …… 21

表 4.3 逻辑二进制区段长度计算 …… 21

表 5.1 示例 XML 上的内容查询正例与负例 …… 26

表 5.2 示例 XML 上的路径查询正例与负例 …… 28

表 6.1 10K 行数据的无索引压缩基线 …… 32

表 6.2 不同存储与查询方案的空间占用对比 …… 32

表 6.3 10K warm-cache 查询延迟比较 …… 33

缩略语说明

外文缩略字母	外文全称	中文说明
XML	Extensible Markup Language	可扩展标记语言
XPath	XML Path Language	XML 路径语言
GIN	Generalized Inverted Index	通用倒排索引
Bloom Filter	Bloom Filter	布隆过滤器
SQL	Structured Query Language	结构化查询语言

第一章　绪论

1.1　研究背景与意义

1.1.1　XML 数据的特征与数据库应用背景

可扩展标记语言（Extensible Markup Language，XML）通过元素及其嵌套关系表达层次化数据，长期用于跨系统数据交换、业务文档、配置数据和异构系统之间的信息传递。XML 1.0 规定了 XML 文档的标记语法与基本良构性要求，但不规定数据库必须采用何种物理存储或索引方式 [1]。XML 的自描述特征使数据结构可以随应用变化，同时也使标签名称、层次路径和文本值在同一文档或多个文档之间反复出现。

与关系型表中固定模式的记录相比，XML 文档具有明显的层次性、顺序性和结构可变性。元素的嵌套关系、节点顺序、空元素以及文本内容共同构成文档语义。将 XML 放入关系型数据库统一管理时，数据库不仅需要保存文档内容，还需要在存储、查询、索引和类型转换之间维持一致语义。因此，数据库 XML 类型的物理表示不能只关注字节数，还需要考虑压缩之后的结构和值信息是否仍然能够参与查询。

PostgreSQL 为 XML 类型提供 xpath、xpath_exists 等查询函数，并通过 libxml2 执行 XPath 1.0 表达式 [2][3]。这为数据库 XML 查询提供了较为成熟的实现基线，也为研究压缩表示如何接入数据库类型和索引机制提供了可扩展的平台。本文以 PostgreSQL 的 XPath 1.0 查询语境为应用背景，围绕路径存在、路径值获取和精确内容匹配等查询意图，研究 XML 压缩表示与查询、索引处理之间的协同关系。

1.1.2　XML 存储与查询的现实需求

XML 存储与查询之间存在若干相互关联的需求。首先，XML 的层次结构不能简单等同于无序的键值集合。将 XML 压平为若干路径和值，便于去重和查询，但如果没有额外结构信息，可能无法准确恢复父子边界、重复容器、节点顺序或空元素。其次，压缩和查询可能重复处理同一语义信息。写入时解析器已经识别了元素路径和文本值；如果查询时仍从原始 XML 重新解析这些信息，压缩阶段形成的中间结果就没有被查询复用；如果在原始 XML 之外另行物化查询数据，又会增加派生数据、索引维护和写入成本。

不同查询所需的信息也并不相同。在 XPath 1.0 语境下，路径存在性需要判断某条路径是否出现；路径值获取需要沿文档顺序找到匹配路径对应的值；内容查询则需要对文本值进行精确判断。因而，XML 压缩方法需要明确编码表示能够直接提供哪些信息、查询过程需要解码哪些信息，以及候选结果如何进行精确验证。索引优化还要同时考虑候选生成和结果正确性：Bloom Filter 可以快速排除部分不可能的候选，但不能把“可能存在”直接当作真实命中；哈希标识也可能发生碰撞，必要时必须回到压缩值或原始值进行精确复核。

1.1.3　研究意义

本文研究如何复用 XML 压缩编码过程中形成的路径标识、值标识和结构记录，使它们在压缩表示生成后继续服务于路径判断、路径值获取、内容匹配及索引键提取。本文强调的不是将 XML 当作普通文本进行压缩，而是利用 XML 的结构特征识别重复路径、重复值和节点关系，形成结构感知的压缩表示；在此基础上，使编码阶段已经产生的中间数据继续被后续查询和索引处理使用。

本研究的意义主要体现在两个方面。方法层面，建立从 XML 解析、结构感知编码、中间数据组织到查询与索引复用的连续处理过程，分析同一组编码信息如何减少后续查询中的重复识别，并明确候选过滤、精确验证和结构恢复之间的正确性关系。应用层面，在 PostgreSQL 扩展原型中将该方法落实为二进制表示、XPath 导向查询函数和 GIN 索引处理流程，再通过实验观察其存储开销、查询表现及写入代价。PostgreSQL 为方法提供 XML 查询基线和可扩展索引环境，使本文能够在数据库类型层面对方法的可行性和基本能力进行验证。

1.2　国内外研究现状

1.2.1　XML 压缩方法研究现状

XML 文档包含重复出现的标签、路径及文本值，相关压缩研究因此从通用文本压缩逐步发展出利用 XML 结构特征的表示方法。XMill 将 XML 内容按结构特征分组，再结合通用压缩技术处理同类数据，体现了结构信息参与压缩组织的思路 [5]。国内 XSC 等工作从 XML 数据流和结构事件出发组织编码，并关注压缩数据的访问与 XPath 处理 [4]。这类方法表明，XML 压缩并非只能把文档视为普通字符序列；元素路径、节点类

别和文本内容都可以影响数据组织及编码方式。

从已有研究可以看到，XML 压缩通常需要在压缩率、解码粒度和结构保真之间进行权衡。面向压缩率的表示可以通过字典、统计编码或通用压缩器减少字节数，但查询时可能需要恢复较大范围的数据；结构感知方法能够利用路径和节点关系减少结构性冗余，但需要额外保存解码和导航所需的结构信息。字典作用域、编码单位、结构事件表示和解码粒度都会影响最终的空间与访问成本。

1.2.2 可查询 XML 压缩研究现状

随着数据库对压缩数据直接访问的需求增加，研究进一步形成了可查询 XML 压缩路线。XGrind 和 XPRESS 研究在压缩表示上进行内容或结构查询的方法 [6][7]；Buneman、Grohe 与 Koch 讨论在压缩 XML 骨架上执行路径查询的方法 [8]。这些工作表明，压缩表示可以保留或构造查询可使用的结构与内容信息，查询不必在所有情况下都先完整恢复原始文档。

后续研究进一步把查询负载和访问粒度纳入压缩数据组织。XQueC 根据查询需求选择结构与数据容器的组织方式 [9]；XSAQCT 关注 XML 压缩数据的随机访问及查询能力 [10]；XQzip 利用结构索引辅助压缩 XML 查询 [11]；XBzipIndex 研究压缩 XML 的导航和搜索 [12]；XXS 与 SXSI 则探索压缩文档或自索引结构上的 XPath 查询 [13][14]。这些方法在支持的查询语言、编码格式、索引形式、压缩单位及查询时解码范围方面各有特点，共同推动了 XML 压缩研究从“压得更小”扩展到“压缩后如何定位并处理查询所需的信息”。

1.2.3 XML 查询与索引研究现状

XML 查询研究包括对文档树进行路径导航、对节点内容进行匹配，以及借助索引减少候选范围等方向。DataGuides 使用结构摘要描述半结构化数据中的实际路径 [15]；IndexFabric 将路径组织为索引键以支持半结构化数据检索 [16]；XISS/R 则研究在关系数据库环境中存储与索引 XML[17]。针对压缩 XML，XQzip、XBzipIndex、XXS 和 SXSI 等工作把结构访问与压缩表示或索引组织结合起来 [11]–[14]。

在数据库实现方面，PostgreSQL 提供 XML 类型以及基于 libxml2 的 XPath 1.0 查询函数 [2][3]，并通过扩展索引接口支持不同数据类型定义键抽取、候选匹配和结果复核机制 [19]。GIN（Generalized Inverted Index，通用倒排索引）可以将一个复合值拆解为

多个索引键，在查询阶段先定位候选行，再由一致性函数和执行器完成必要的复核。已有研究和系统共同说明，XML 查询可以由文档内结构访问、跨文档索引和数据库执行器协作完成；同时，索引还会带来额外的存储、构建和维护成本。

1.2.4　研究现状小结

总体来看，国内外研究已经形成结构感知压缩、可查询压缩、路径索引、内容检索和数据库集成等多条技术路线。现有工作为本文提供了重要基础：XML 的结构和内容可以在编码阶段被分别识别和组织，压缩表示可以保留查询所需的信息，结构索引和倒排索引可以减少无关访问，数据库扩展接口可以将自定义表示接入查询执行过程。

在这些研究基础上，本文进一步关注编码阶段信息在多个后续环节之间的复用关系：路径和值在编码时被识别之后，能否以一致表示同时服务于压缩存储、XPath 导向查询和索引键提取；为此保留的结构和辅助信息会带来怎样的空间及处理代价；Bloom Filter 或哈希候选如何与精确验证结合以保持查询结果正确。上述问题构成本文研究目标与研究内容的出发点。

1.3　研究目标与研究内容

1.3.1　研究目标

本文的总体目标是面向数据库 XML 类型，研究一种以结构感知压缩和压缩编码复用为核心的压缩与索引优化方法。该方法在 XML 解析阶段识别元素路径、文本值和节点结构，将可重复的路径和值映射为可复用标识，并组织为可解码的二进制压缩表示；在查询和索引处理阶段继续消费这些编码标识，从而减少对同一 XML 语义信息的重复解析和重复识别。

围绕总体目标，本文重点解决以下研究问题：

（1） 如何构造同时保留 XML 结构、文档顺序和内容信息的压缩表示，使结构感知压缩不会破坏节点语义和解码正确性；

（2） 如何使压缩编码阶段形成的路径 ID、文本 token ID 和结构记录被压缩表示、XPath 导向查询函数及索引键提取共同复用；

（3） 如何将路径存在、路径值获取和精确内容匹配等查询过程分解为候选定位与精确验证，并明确 Bloom Filter、路径索引和 GIN 索引各自的作用范围；

（4） 在统一统计口径下，如何分别评价压缩列值、完整 relation、索引和共享字典的空间开销，以及查询性能和写入代价。

本文以 PostgreSQL 的 XML 类型和 XPath 1.0 查询能力作为应用语境和实现基线。原型围绕上述查询意图实现相应函数和索引接口，研究结论限定在当前方法、原型、数据集和测试配置范围内。

1.3.2 研究内容

围绕上述目标，本文开展以下四项相互衔接的研究工作。

（1） **压缩表示与查询操作的语义定义。**对 XML 解析所得的路径、叶文本、节点顺序和结构关系进行形式化表示，明确路径存在、路径值获取和精确内容匹配等查询操作的输入、输出及正确性条件。

（2） **面向查询的结构感知压缩方法。**复用编码阶段形成的路径 ID 和文本 token ID，将其组织为文档序内容流、排序路径索引、Bloom 元数据、长值内联区和结构流；短值使用字典标识，整数使用 Varint 编码，同时保留恢复 XML 结构所需的信息。

（3） **基于压缩表示的查询与索引方法。**在文档内部利用路径索引和内容流完成路径判断、路径值获取及精确内容查询，在关系级利用 GIN 生成候选行；对于 Bloom Filter 假阳性和长值哈希碰撞，均通过精确扫描或谓词 recheck 保证结果语义。

（4） **原型实现与实验评估。**在 PostgreSQL 扩展中实现自定义数据类型、输入输出函数、查询函数和 GIN 操作类，通过编码/解码正确性测试、空间测量、查询性能测试和消融观察评估方法的可行性与代价边界。

1.3.3 主要贡献

本文的主要贡献可以概括为以下三点：

（1） 提出以压缩编码中间数据复用为核心的 XML 压缩与索引优化方法，将编码阶段形成的路径和值标识组织为可被压缩存储、查询处理和索引键提取共同消费的中间表示；

（2） 建立结构感知压缩表示与查询处理之间的对应关系，在保留文档序内容和节点结构的同时，结合路径索引、Bloom Filter、GIN 候选定位及精确验证支持 XPath 导向查询；

（3） 在 PostgreSQL 扩展原型中验证方法的可实现性，并从列值、relation、索引、共享

字典、查询和写入等多个维度报告实验观察，明确不同统计口径和实验条件下的适用边界。

本文的创新主张针对上述方法流程及其正确性约束，不将字典编码、Varint、Bloom Filter 或倒排索引单独宣称为新算法，也不把当前原型实验结果泛化为对其他数据库平台或所有可查询 XML 压缩系统的普遍性能结论。

1.3.4　研究范围与设计要求

本文研究范围是数据库 XML 类型的压缩表示、查询处理和索引优化方法，重点考察结构感知压缩和编码中间数据复用。方法设计遵循以下要求：一是利用 XML 的结构和内容特征进行压缩，避免把 XML 仅作为普通文本字节串处理；二是使编码阶段形成的路径和值标识供压缩、查询和索引键提取复用；三是保留文档顺序和结构信息，保证解码和查询语义；四是区分候选过滤与精确结果判定，避免 Bloom Filter 假阳性和哈希碰撞造成错误命中；五是分别报告空间、查询和写入代价，不用单一指标替代完整评价。

1.4　论文组织结构

1.4.1　技术路线

本文的技术路线如下：首先，从 XML 数据结构和数据库查询需求出发，分析结构性冗余、压缩后查询和索引维护之间的关系，确定编码中间数据复用这一核心问题；其次，对 XML 文档进行语义解析，提取路径、文本值、节点顺序和结构关系，并通过字典及变长整数编码形成结构感知的压缩表示；再次，利用同一组路径和值标识构造路径索引、Bloom 摘要和 GIN 索引键，使查询处理能够直接消费编码阶段形成的信息；最后，通过正确性测试以及存储、查询、写入和消融实验评价方法能力，分析方法的适用条件与有效性威胁。

1.4.2　章节安排

全文共分为七章，各章内容如下：

第 1 章 绪论。介绍 XML 数据库应用背景、XML 存储与查询需求、国内外研究现状，提出本文研究目标、研究内容、主要贡献和技术路线。

第 2 章 XML 压缩存储与查询的理论基础。介绍 XML 数据模型、结构感知压缩、可查询 XML 压缩、路径与内容索引、字典编码、Varint、Bloom Filter 以及 PostgreSQL 扩

展和 GIN 机制。

第 3 章 可查询 XML 压缩方法及问题分析。基于前人研究分析编码阶段信息复用、查询候选生成与精确验证，以及压缩、查询和索引代价之间的耦合关系，明确本文的方法要求。

第 4 章 面向查询的 XML 压缩表示方法。给出方法总体思路、XML 结构与内容编码、字典复用、变长整数编码、辅助数据组织，以及压缩表示的解码和正确性约束。

第 5 章 基于压缩表示的查询处理方法。说明查询模型、精确内容查询、路径存在与值提取、路径索引与 Bloom 过滤、候选验证以及 PostgreSQL 操作符和 GIN 索引实现。

第 6 章 实验评估与结果讨论。围绕研究问题设计实验，评估编码正确性、存储开销、查询性能、索引效果和写入代价，并讨论实验结果的适用范围与有效性威胁。

第 7 章 总结与展望。总结本文工作和主要结论，说明研究不足，并提出后续研究方向。

1.4.3 本章小结

本章从 XML 数据特征和数据库应用需求出发，说明了 XML 存储、压缩、查询和索引之间的关联，概述了国内外 XML 压缩、可查询压缩以及 XML 查询索引研究，并在此基础上提出本文的研究目标、研究内容和主要贡献。本文的核心研究主张是：以结构感知方式压缩 XML，并复用压缩编码阶段产生的路径、值和结构信息，使其继续服务于 XPath 导向查询和索引键提取。下一章介绍理解该方法所需的理论基础与代表性技术路线。

第二章　XML 压缩存储与查询的理论基础

2.1　XML 数据模型及存储特征

XML 文档可抽象为有序树。元素节点具有名称和子节点序列，文本节点承载字符内容；节点顺序属于文档语义的一部分。数据库中的 XML 查询需要同时处理结构信息与内容信息：结构信息包括从根到节点的路径、节点层次及其先后关系，内容信息包括文本值及其所在节点。本文方法将叶元素的完整路径作为查询和编码的基本结构单位，并保留完整节点结构流以支持结构恢复。

数据库 XML 列以逻辑 XML 值为存取对象，但物理存储还可能涉及页内表示、超大属性存储技术（The Oversized-Attribute Storage Technique，TOAST）及数据库版本相关的压缩行为 [20]。因此，分析存储成本时应区分列值、表与 TOAST relation、索引以及数据库外共享字典，不能将列值大小直接等同于系统总空间。

2.2　XML 压缩方法

XML 压缩方法利用标签、路径和文本内容中的重复性降低表示开销。通用字典或统计压缩把 XML 作为字节序列处理；结构感知方法则在编码前解析标记、路径或节点事件，并据此分离或组织不同类别的数据。XMill 将 XML 数据按结构特征分组后进行压缩，体现了结构参与数据组织的思路 [5]。XSC 等 XML 数据流压缩研究则从结构事件和流式编码出发，将 XML 结构处理与数据编码联系起来 [4]。

这类方法的压缩对象可以包括重复标签、完整路径、文本值及树结构。若把重复路径映射到字典标识，再将标识编码为整数，可减少结构字符串的重复存储；若把相同文本值映射为 token 标识，则可在重复值较多时减少内容占用。不同方法的字典作用域、编码单位和解码粒度不同，压缩率也会受到数据重复度及结构相似度影响。结构感知压缩仍须保存足以恢复节点顺序和边界的信息，否则紧凑表示可能无法重建原始树形语义。

2.3 可查询 XML 压缩方法

可查询 XML 压缩在压缩表示的设计中考虑查询所需的信息，使查询处理有机会避免先完整恢复原文。XGrind 通过对 XML 结构和内容进行编码，研究压缩数据上的精确匹配、前缀匹配和相关查询过程 [6]。XPRESS 研究 XML 数据的可查询压缩表示 [7]。Buneman、Grohe 与 Koch 讨论在压缩 XML 骨架上执行路径查询的方法 [8]。这些工作表明，压缩表示可以保留或构造查询可使用的结构与内容信息。

后续研究进一步把查询负载和访问粒度纳入压缩数据组织。XQueC 根据查询需求选择结构与数据容器的组织方式 [9]；XSAQCT 关注 XML 压缩数据的随机访问及查询能力 [10]。XQzip 利用结构索引辅助压缩 XML 查询 [11]，XBzipIndex 研究压缩 XML 的导航和搜索 [12]，XXS 与 SXSI 则探索压缩文档或自索引结构上的 XPath 查询 [13][14]。这些方案在支持的查询语言、编码格式、索引形式及查询时解码范围方面各有特点，不能视为同一种算法或直接等价的性能基线。

从方法角度看，可查询压缩的关键不只是“压缩后能否查询”，还包括查询直接消费什么信息、候选如何定位、查询需要解码的范围，以及辅助信息由何种阶段构建。不同研究对上述问题采用了不同取舍，这为后续分析编码信息的复用方式提供了基础。

2.4 XML 路径查询与索引方法

XML 路径查询依据元素层次选择节点或判断节点存在。路径索引研究尝试将路径结构转化为可检索的键或摘要。DataGuides 使用数据结构摘要描述半结构化数据中的实际路径 [15]；IndexFabric 将路径组织为索引键以支持半结构化数据检索 [16]；XISS/R 研究在关系数据库环境中存储与索引 XML[17]。针对压缩 XML，XQzip、XBzipIndex、XXS 和 SXSI 等工作把结构访问与压缩表示或索引组织结合起来 [11]–[14]。

路径索引能够降低查询中与目标路径无关的访问，但索引键的生成、存储和更新也构成额外成本。路径存在判断、路径对应值的提取以及跨文档内容检索并非同一操作：路径存在可由路径摘要回答；取值还需要关联文档顺序及内容记录；跨文档内容查询则需要在记录集合层面生成候选。因此，索引结构须与查询语义和数据表示相匹配。

2.5　XML 内容查询与候选过滤

内容查询可以采用精确字符串判断、词项检索或模式匹配等不同语义。若方法使用字典将文本值映射为标识，精确 token 查询可以先将查询值映射到同一标识，再在压缩内容记录或索引中定位候选。对长文本使用哈希值时，哈希相等只能说明候选可能相同，必须通过原文比较排除碰撞。

Bloom Filter 是基于位数组和多个哈希函数的集合近似结构。任一探测位为零时可以确定元素不存在；所有探测位均为一时只能判为可能存在，仍需要精确检查。它适合作为候选否定过滤器，而不能单独作为精确查询结果 [22]。压缩文档中的 Bloom 摘要、关系级倒排索引和精确谓词各自作用范围不同：前者可在单文档内否定候选，倒排索引可跨文档定位候选行，最终判断仍由精确语义负责。

2.6　字典编码、Varint 与 Bloom Filter

字典编码把重复字符串映射为整数 ID。若相同字符串在指定字典作用域内复用同一 ID，内容流便可用整数标识替代重复字符串，查询输入也可转换为相同标识进行比较。收益取决于重复率、字典作用域和字典自身的存储成本。跨文档共享字典可以摊销重复字符串的空间，但字典并非零成本。

变长整数编码（Varint）根据整数大小采用不同字节数。本文实现采用 LEB128（Little Endian Base 128）风格编码：每字节低 7 位承载有效位，最高位作为延续标记。较小 ID 通常使用较少字节，同时保持顺序解码。相关整数压缩研究讨论了紧凑表示与快速访问之间的权衡 [21]；Varint 是本文采用的编码选择，不是本文单独提出的整数压缩算法。

Bloom Filter 使用位数组和多个哈希探测近似判断集合成员。若任一探测位为 0，则目标一定不存在；若所有探测位均为 1，只能判为“可能存在”，后续仍需精确检查。在构建与查询使用一致参数和标识的前提下，Bloom Filter 允许假阳性而不允许假阴性 [22]。本文中，字典 ID 提供可复用查询键，Varint 控制这些键在内容流中的编码成本，Bloom Filter 以 token ID 为元素构造文档级摘要，最终结果仍由内容记录确认。

2.7 PostgreSQL 扩展机制与 GIN 索引

PostgreSQL 允许扩展注册自定义数据类型、输入输出函数、操作符及索引操作类。本文原型使用自定义类型承载压缩表示，并实现相应查询函数与操作符。数据库扩展机制使特定二进制格式能够参与 SQL 查询，但正确性仍取决于类型输入输出、操作符语义及索引回调之间的一致约定。

GIN（Generalized Inverted Index，通用倒排索引）面向可由多个键表示的复合值。索引构建阶段，操作类从数据项提取键；查询阶段，将查询值转换为键并从倒排结构获取候选；一致性回调判断候选是否需要复核 [19]。键抽取与索引维护本身存在成本。本文原型的 GIN 键来自压缩内容流中的 token ID；长 token 的哈希仅用于候选产生，精确字符串比较用于排除碰撞。Bloom Filter 不作为 GIN 键来源，二者不能混为同一种索引结构。

2.8 本章小结

本章介绍了理解本文方法所需的 XML 数据模型、结构感知压缩、可查询 XML 压缩、路径与内容索引、字典编码、Varint、Bloom Filter 及 PostgreSQL GIN 机制，并梳理了相关研究形成的主要技术路线。下一章不再重复介绍这些方法，而是从编码信息如何流向查询与索引、候选如何得到精确验证及其代价如何衡量等角度，分析本文研究问题的来源。

第三章　可查询 XML 压缩方法及问题分析

3.1　分析对象与比较维度

第二章回顾的结构感知压缩、可查询压缩和 XML 索引研究，分别回答了如何利用 XML 结构组织压缩数据、如何在压缩表示上处理查询，以及如何借助索引减少访问范围等问题 [4]–[17]。本章在这些成果基础上分析本文的研究对象，不再逐项介绍已有方法，而是聚焦一个贯穿编码、查询和索引构建的过程问题：编码阶段已经识别出的结构和值信息，能否以一致表示留存并被后续处理复用；复用这些信息所需的辅助空间及处理成本又如何衡量。

为分析该问题，本文从四个维度考察方法链：第一，信息形成，即 XML 解析过程中识别了哪些路径、值和节点关系；第二，信息留存，即哪些识别结果进入压缩表示或辅助结构；第三，信息消费，即查询函数和索引键抽取是否使用同一语义标识；第四，正确性约束，即摘要和哈希产生候选后如何恢复精确判断。该分析维度将压缩率、查询时间和索引成本置于同一处理链中，而不把任何单一组件孤立地视为方法贡献。

表 3.1 将第二章介绍的代表性工作映射到上述分析维度。表中“本文分析重点”是本文据其技术路线提出的比较视角，并非对原论文实验结果的重新评价。

表 3.1　代表性 XML 方法与本文的分析维度

代表性方法	相关研究关注点	本文据此分析的环节
XMill、XSC[4][5]	利用 XML 结构或事件组织压缩数据	解析阶段识别的结构信息如何进入压缩表示
XGrind、XPRESS[6][7]	在压缩 XML 表示上支持查询处理	查询需要消费哪些压缩信息及其访问粒度
XQueC、XSAQCT[9][10]	根据查询需求、随机访问调整数据组织	查询能力与压缩单位、额外组织成本的关系
DataGuides、IndexFabric、XQzip、SXSI[11][14]–[16]	以结构摘要、路径键或压缩索引支持导航与检索	路径信息在文档内定位和跨文档索引中的职责

3.2 编码阶段信息复用的分析

XML 编码通常先经过解析或结构识别，再执行具体压缩。设输入文档为 D，解析阶段从中提取路径集合 P_D、文档序内容记录 R_D 和节点结构记录 T_D。若这些结果只作为编码时的临时数据，压缩后丢弃，则后续查询需要从原始文档重新解析，或先解压再提取查询所需信息。若为查询另外建立派生结构，则需要重复识别路径和值，并承担独立存储及一致性维护成本。

另一种组织方式是令编码结果成为多个阶段共享的中间表示。对字符串路径 p 和文本值 v 分别定义映射 $\phi_p(p)$ 与 $\phi_v(v)$，将路径和值转换为标识；压缩内容流、路径辅助信息及索引键提取均消费这些标识。其直接作用不是保证每个步骤都免于扫描，而是避免对同一语义对象重复执行字符串识别和独立编码。特别是，GIN 建索引仍须读取压缩值并提取键；信息复用减少的是重新解析 XML 和重新定义 token 语义的需要，不能表述为零成本索引构建。

从这一分析可得出信息复用需要满足的三个条件：其一，编码端和查询端的标识映射具有一致的字典作用域；其二，压缩流保留查询及解码所需的顺序和结构关系；其三，派生的路径索引、Bloom 摘要和关系级倒排项具有明确的职责边界。若只保留叶路径和值，内部容器及节点边界可能不足以恢复；若只保留结构摘要，则又不能直接完成精确内容判断。因此，本文方法同时分析内容序列与结构序列，而不是把 XML 压平为无序键值集合。

3.3 查询候选生成与精确验证分析

查询处理可以分解为候选生成和精确判定两个阶段。令目标查询为 q，其真实匹配文档集合为 $A(q)$，候选生成阶段返回集合 $C(q)$。正确的过滤结构应满足 $A(q) \subseteq C(q)$，即可以多返回候选，但不能漏掉真实匹配；精确验证再从 $C(q)$ 中筛出 $A(q)$。这一关系适用于 Bloom Filter 及基于哈希的索引候选，但二者的使用范围不同。

在单文档内部，路径存在判断可由路径标识及文档级路径索引定位；路径取值还需要访问与该路径关联的文档序内容记录。Bloom Filter 针对内容值提供快速否定：若摘要判断不存在，可跳过该文档的精确内容扫描；若判断可能存在，则必须扫描内容标识或原文进行精确比较。由于 Bloom Filter 允许假阳性，阳性结果不是命中结论。在跨文

档层面，GIN 根据内容键定位候选行；当长值以哈希键表示时，哈希碰撞可能引入额外候选，执行器仍需 recheck。

由此可以区分三种容易混淆的成本和作用：路径索引回答单个压缩文档中是否存在目标路径；文档级 Bloom 摘要减少单文档内容扫描；GIN 倒排结构减少关系中需要访问的行。它们可以协作，却不能互相替代。查询速度提升取决于候选缩减比例、精确复核成本、缓存与数据库执行器开销；仅由某一过滤结构的理论性质不能推出端到端延迟必然下降。

3.4　压缩、查询与索引代价的耦合关系

压缩与查询共享信息会形成空间—处理的耦合。设原始 XML 逻辑列值总空间为 S_X，压缩列值为 S_C，辅助表示为 S_A，关系级索引为 S_I，共享字典分摊空间为 S_D，则在某一明确统计口径下，完整持久化成本可写为

$$S_{\text{total}} = S_C + S_A + S_I + S_D.$$

实际数据库 relation 统计可能已包含部分列值或 TOAST 开销，因此比较时必须按同一物理口径避免重复相加。该式强调：压缩列值变小并不自动推出含索引系统总空间变小；共享字典、GIN 和其他关系开销都应在同一边界内核算。

查询成本可按处理阶段表示为

$$T_q = T_{\text{lookup}} + T_{\text{candidate}} + T_{\text{scan}} + T_{\text{verify}} + T_{\text{executor}},$$

其中各项分别表示查询标识映射、候选生成、压缩表示扫描、精确复核和数据库执行器开销。复用编码标识的目标是减少重复解析与字符串识别成本；Bloom 或 GIN 可能减少后续扫描量，但它们自身也有维护、访问和复核成本。因而，方法的实际效果应通过查询测量而非单项复杂度推断。

数据特征也影响压缩收益。设总叶记录数为 N，不同路径数为 U_p，不同短文本数为 U_v，结构模板数为 U_s。当文档数量增加而 U_p、U_v 的增长较慢时，重复路径和值能被字典复用的次数增加；当单文档内容更密集时，固定头部、路径元数据及结构边界成本可由更多记录摊薄；当文档结构相似时，路径和节点关系的重复程度提高，结构标识更容易复用。因此，在字典和辅助信息策略相同的条件下，这些性质为压缩比随数据规模、密度和结构相似性提高而改善提供了机制解释。该判断是基于编码成本的趋势分

析，仍需通过多组数据测量验证，不能替代实验结果。

3.5 本文研究问题、适用范围与设计要求

上述分析将本文研究问题归纳为：如何在 XML 结构感知压缩过程中复用产生的路径和值标识，使压缩表示、XPath 导向查询处理及索引键提取共享一致的信息基础，同时保留解码和精确验证所需的结构信息，并对由此产生的空间与查询代价进行可比评价？PostgreSQL 是本文实现和验证方法可行性的数据库平台；研究对象是压缩与索引优化方法，平台实现用于观察方法在数据库类型中的落地能力，而不是将研究贡献限定为 PostgreSQL 某种系统设计。

本文方法和实验遵循以下要求：一是使用结构感知表示压缩 XML，避免把层次化数据仅视为普通文本；二是令编码阶段产生的路径和值标识供压缩、查询及索引键抽取复用；三是明确路径判断、路径取值和内容查询的语义，并保留文档顺序及结构信息；四是将 Bloom 或哈希候选与精确结果判定分开；五是分别报告压缩列值、索引、relation、共享字典等空间口径，以及查询和写入测量的条件；六是把实验结论限定在当前原型、数据集和配置，不据此宣称对所有可查询 XML 压缩方法或数据库平台具有普遍优势。

3.6 本章小结

本章从作者的研究问题出发分析了编码信息复用的必要条件、候选生成与精确验证的关系，以及压缩、查询和索引成本之间的耦合。分析表明，方法核心在于以一致的编码标识连接压缩表示、查询处理和索引键抽取，同时用结构信息支持解码、用精确复核约束候选结果。第四章据此给出压缩表示及编码过程，第五章说明基于该表示的查询处理方法。

第四章　面向查询的 XML 压缩表示方法

4.1　方法总体思路与设计目标

本文方法以“编码阶段识别的信息继续服务于查询阶段”为主线。输入 XML 经语义解析后形成叶路径—文本记录和完整节点结构记录；编码过程将路径和值映射到相应字典标识。方法不把这些标识仅视为压缩字节流的临时产物，而是使同一标识契约贯穿压缩值构造、查询判断和索引键提取，避免各阶段重复识别同一语义对象。索引构建仍需扫描并解码压缩内容流，因此“复用”不等于消除索引键提取成本。

设 D 为符合原型输入约束的 XML 文档。解析后得到文档序叶记录

$$R_D = \langle (p_i, v_i) \rangle_{i=1}^{n}$$

和前序节点记录

$$T_D = \langle (t_j, c_j) \rangle_{j=1}^{m},$$

其中，p_i 是叶元素的完整路径，v_i 是其解析后文本，t_j 是节点路径，c_j 是子节点数。路径和值字典分别定义映射 ϕ_p 与 ϕ_v；在当前字典生命周期内，每个已注册路径或短值映射到唯一 ID。值标识函数定义为

$$\eta_v(v) = \begin{cases} \phi_v(v), & |v|_{\text{UTF8}} < 1024, \\ \text{tag}(H_{32}(v)), & |v|_{\text{UTF8}} \geq 1024. \end{cases}$$

长值哈希仅用于候选定位，不能替代原文比较。编码中间表示为

$$M(D) = (E_D, P_D, F_D, L_D, S_D), \qquad B_D = \text{serialize}(M(D)),$$

其中 $E_D = \langle (\phi_p(p_i), \eta_v(v_i)) \rangle$ 是保留文档顺序的内容序列；P_D 是由节点路径及叶记录数构成的有序路径摘要；F_D 是由内容值标识构成的 Bloom 摘要；L_D 保存长值内联候选；S_D 是使用同一路径标识表达的前序结构序列。编码器、查询算子和索引键提取器对同一受支持路径或短文本使用同一字典映射；长文本哈希只作候选标识，之后仍须精确复核。

该方法可由三项性质检验：同一语义值在规定字典作用域内具有一致 ID；派生摘要只减少候选判断工作而不改变精确结果；结构与内容两类编码信息共同支持所定义

语义下的解码。处理流程如图 4.1 所示。



图 4.1 XML 编码与中间数据复用流程

方法设计目标包括：将路径和值冗余压缩为可复用标识；保留解码所需的顺序和结构；使目标查询能够消费表示中的索引或内容区；明确候选、精确验证和外部字典各自的边界；避免把局部有效的组件特性夸大为端到端性能保证。路径索引、Bloom 元数据及 GIN 键提取共享编码阶段的 ID，但各自作用范围不同。字典的稳定作用域是当前 PostgreSQL 数据目录，不表示跨集群 ID 永久一致。

** 方法过程 1：编码中间表示复用。** 输入为 XML 文档 D 及路径、短 token 字典 Δ_p, Δ_v；输出为压缩值 B_D 及其解码所需的结构信息。伪代码如下。

代码 4.1: 编码中间表示复用过程

```
R, T <- parse_once(D)
E <- empty ordered content sequence
PairCount <- empty path-to-leaf-pair-count map
F <- empty Bloom filter; I <- empty inline-token map

for each node in preorder records T:
    pid <- lookup_or_assign(Delta_p, node.path)
    PairCount.setdefault(pid, 0)

for each (path, value) in document-ordered leaf records R:
```

```
    pid <- lookup_or_assign(Delta_p, path)
    if UTF8_byte_length(value) < 1024:
        vid <- lookup_or_assign(Delta_v, value)
    else:
        vid <- tagged_inline_hash(value)
        if I contains vid and I[vid] != value: reject document
        I[vid] <- value
    append (pid, vid) to E
    PairCount[pid] <- PairCount[pid] + 1
    insert vid into F

S <- encode preorder records T using the same path IDs
Pidx <- sort PairCount by pid
B_D <- serialize(header, Pidx, E, I, F, S)
return B_D
```

这里的“复用”有明确含义：路径索引及 Bloom 由解析和编码时形成的内容信息派生；GIN 建索引时从序列化压缩内容中提取同一 token ID，查询侧再按相同字典映射查询字符串。因此，GIN 的键提取仍有扫描压缩流的成本，但不必重新解析原始 XML 或另行推断 token 边界。

设文档含 n 条叶记录、u 个不同路径索引项，Bloom 使用 k 次位探测。排序路径索引上的路径存在查询需要 $O(\log u)$ 次比较；路径取值还要扫描内容流，最坏为 $O(\log u + n)$；Bloom 给出否定结果时需 $O(k)$ 次探测，可能阳性或关闭 Bloom 时仍可能扫描 $O(n)$ 条内容记录；GIN 为单个压缩值提取键也需解码其内容记录，成本随 n 增长。上述渐进界不包括解析、字典查找、序列化及数据库 I/O 的端到端成本。

4.2　XML 结构与内容的编码表示

4.2.1　贯穿本章的 XML 示例

为了说明 XML 从文本到压缩表示的转换过程，本文使用如下 XML 文档作为贯穿本章的示例。该文档包含重复的 item 容器、重复的 name 和 quantity 路径，能够体现结构感知压缩和文档序信息保留的作用。

代码 4.2: 用于说明编码过程的 XML 示例

```
<orders>
  <order>
    <id>1001</id>
    <customer>张三</customer>
    <item>
      <name>机械键盘</name>
      <quantity>2</quantity>
    </item>
    <item>
      <name>鼠标</name>
      <quantity>1</quantity>
    </item>
  </order>
</orders>
```

解析器维护当前路径栈。遇到开始标签时压入标签，遇到结束标签时弹出标签；遇到叶节点文本时，记录当前完整路径和文本值。由此得到文档序叶记录

$$R_D = \langle (/\text{orders/order/id}, 1001), (/\text{orders/order/customer}, \text{张三}), (/\text{orders/order/item/name}, \text{机械键盘}) \rangle$$

该顺序与 XML 中叶节点的出现顺序一致，因而可以支持路径取值的首值语义，也可以在解码时恢复叶节点次序。

仅有 R_D 仍不足以恢复完整 XML 树。为此，解析器同时生成前序结构记录 T_D，其中每条记录由节点路径和直接子节点数构成。例如，根节点 /orders 的子节点数为 1，/orders/order 的子节点数为 4，两个重复的 /orders/order/item 节点的子节点数均为 2，而叶节点的子节点数为 0。解码器按前序读取结构记录；读取到内部节点时创建容器节点，读取到叶节点时消费内容流中的下一条记录。这样，内容流负责保存文本及其顺序，结构流负责保存节点边界和嵌套关系。

4.2.2 路径和值的标识化

对上述示例，在一个确定的字典作用域内，可以建立如下路径字典和短文本字典。表中的具体整数仅用于说明编码过程，实际 ID 取决于字典注册顺序。

表 4.1　XML 示例的路径和值标识

路径或文本值	类型	标识
/orders	path	1
/orders/order	path	2
/orders/order/id	path	3
/orders/order/customer	path	4
/orders/order/item	path	5
/orders/order/item/name	path	6
/orders/order/item/quantity	path	7
1001	token	1
张三	token	2
机械键盘	token	3
2	token	4
鼠标	token	5
1	token	6

因此，叶记录可以由字符串对转换为整数对：

$$R'_D = \langle (3,1),(4,2),(6,3),$$
$$(7,4),(6,5),(7,6)\rangle.$$

内容流保存 R'_D 的文档序顺序，而不是按路径排序。路径索引另行保存路径标识及其内容记录数，例如 path_id=6 的记录数为 2，path_id=7 的记录数为 2；内部路径 path_id=5 可以以记录数 0 保留在路径索引中，以支持路径存在判断。路径字典和 token 字典使用独立的 ID 空间，避免路径和文本值在查询语义上混淆。

解析阶段对输入文档进行树遍历，构造有序内容序列：

$$C(D) = \langle (p_1, v_1), (p_2, v_2), \ldots, (p_n, v_n)\rangle,$$

其中 p_i 是叶元素从根到叶的完整路径，v_i 是解析后的文本值，序列顺序与 XML 中对应叶节点的文档顺序一致。内容序列可供查询和内容编码使用，但不足以独立还原所有树结构。因此，方法另生成前序结构序列：

$$S(D) = \langle (\mathrm{path_id}_1, \mathrm{child_count}_1), \ldots, (\mathrm{path_id}_m, \mathrm{child_count}_m)\rangle.$$

每个节点记录其路径标识和子节点数。内部节点按前序记录；叶节点的 $\mathrm{child_count} = 0$ 与内容流中的下一个叶内容记录对应。解码器据结构序列恢复元素边界和子节点次序，并按叶记录填入文本。空叶节点可由空文本记录表示，内部空容器则由结构流本身保

留。

原型采用的逻辑二进制布局见表 4.2。该表描述逻辑区段，不表示所有文档都含有非空 Bloom 或内联区。

表 4.2　逻辑二进制布局及区段内容

区段	内容	长度（B）
PostgreSQL varlena header	PostgreSQL TOAST/varlena 头	4
CXHeader	固定头	32
Sorted path index	路径索引区	$4 + 8P$
Document-order content stream	文档序内容流	C
Optional inline-token section	长值内联区	I
Bloom-filter section	Bloom 区	$8 + \lceil m/8 \rceil$
Preorder structure stream	前序结构流	$4 + S$

表中 P 为路径索引项数，C 为内容流长度，I 为长值内联区长度，m 为 Bloom 位数组长度（位），S 为结构记录编码长度。各区段长度的组成见表 4.3。

表 4.3　逻辑二进制区段长度计算

区段	长度及字段组成
PostgreSQL 头	4 B，用于记录变长值的头部信息。
固定头	32 B，由 8 个 uint32 字段组成：版本、标志、字典 ID、总长度、3 个区段偏移及 Bloom 长度。
路径索引区	$4 + 8P$ B：4 B 的 $\mathrm{path_count}$、4 B 的 $\mathrm{content_pair_count}$，以及 P 个各占 8 B 的 $(\mathrm{path_id}, \mathrm{pair_count})$ 条目。
内容流	$C = \sum_{i=1}^{N} [V(\mathrm{path_id}_i) + V(\mathrm{token_id}_i)]$ B；每条记录只保存两个变长整数 ID。
长值内联区	I B，随内联长 token 的数量及其 UTF-8 字节长度变化。
Bloom 区	$8 + \lceil m/8 \rceil$ B；8 B 保存参数信息，后续为位数组。当前实现取 $m =$ nextpow2(max$(1024, 10N)$)。
结构流	$4 + S$ B，其中 $S = \sum_{j=1}^{M} [V(\mathrm{path_id}_j) + V(\mathrm{child_count}_j)]$；前 4 B 保存节点数。

其中 $V(x)$ 表示对整数 x 进行 Varint 编码后的字节数；N 为内容对数量，M 为节点数量。上述计算刻画逻辑表示长度，不等同于 PostgreSQL relation 的完整物理空间统计。

以表 4.1 的示例为例，文档包含 $P = 7$ 个路径索引项、$N = 6$ 条叶内容记录和 $M = 10$ 个节点。由于示例中的所有标识均小于 128，按当前 Varint 规则每个标识占 1 B，因而内容流长度为 $C = 6 \times (1 + 1) = 12$ B，结构流编码长度为 $S = 10 \times (1 + 1) = 20$ B。该文档没有长值内联区，Bloom 位数组按 $m =$ nextpow2(max$(1024, 10N)) = 1024$ 位计

算，占 128 B，连同 8 B 参数区共 136 B。由此得到的逻辑长度为

$$4 + 32 + (4 + 8 \times 7) + 12 + 0 + 136 + (4 + 20) = 268\ \mathrm{B}.$$

这个数值只用于展示各区段如何共同构成一个压缩值；实际数据库 relation 还会包含行头、对齐、TOAST、索引页和共享字典等开销。

CXHeader 保存版本、标志、字典标识、总长度及相关区段偏移。当前字段名沿用兼容布局：content_offset 指向路径索引区起始位置，path_index_offset 指向后续内容区起始位置，bloom_offset 指向 Bloom 区。内容区之后的内联区按需存在；结构流位于 Bloom 数据之后，可由版本和长度边界定位。

将内容流保持为文档序有两个原因：解码时可按原顺序消费叶记录；路径取值查询可返回文档序上的首个匹配值。相应代价是路径索引仅记录每条路径对应的叶内容记录数，不保存可直接跳转到所有对应值的连续区间；同一路径内容可能交错出现，因此 xml_get 在路径确认后仍需扫描内容流。

4.3　字典复用与变长整数编码

对每个路径字符串，编码器查询路径字典得到 path_id；对短文本值，编码器查询 token 字典得到 token_id。同一作用域内重复字符串复用同一整数标识，路径和 token 使用不同字典，避免语义混淆。字典不仅服务压缩，也为查询提供字符串到 ID 的映射，并为 GIN 查询键保持一致的语义基础。

路径和值标识在内容流中使用 Varint 序列化。Varint 以 7 位为一组编码整数，最高位用于指示是否还有后续字节。较小 ID 通常少于固定 32 位编码的字节数，解码器则顺序读取 $(\mathrm{path_id}, \mathrm{token_id})$ 对。Varint 的作用是降低内容区 ID 表示的空间成本。

当前原型对 UTF-8 字节长度小于 1,024 的 token 使用共享字典；达到或超过阈值的长 token 不进入短 token 全局字典，而写入当前文档内联区。长 token 的 ID 通过置高位的 32 位哈希产生，并与短 token ID 区分。若写入同一文档时两个不同长值发生冲突，当前实现拒绝该输入；跨文档冲突仍可能导致 GIN 产生额外候选，因此查询必须精确复核。路径字典自身也有容量和键长边界。

共享字典不是零成本组件。当前 PostgreSQL 原型在共享内存中维护路径与 token 哈希表，并将字典持久化到数据目录下的独立文件；其空间应计入系统级评价。事务回滚

可能在服务器运行期间留下未被已提交文档引用的字典项，因此本文不声称字典条目与数据行具有完整事务回滚语义。字典 ID 数值取决于注册顺序，也不保证不同实例中一致。这些生命周期事实属于当前原型实现边界，不应与中间标识复用的抽象方法原则混为一谈。

4.4 面向查询的辅助数据组织

排序路径索引。编码器先为结构流中的所有节点路径建立索引项，包括只对应内部节点的路径；再统计叶内容流中各 path_id 的记录数作为 pair_count，最后按 path_id 升序排列。因此，内部节点路径可以在索引中以 pair_count = 0 表示，路径存在查询仍以索引项是否存在为准；pair_count 表示对应叶内容记录数量，不是路径节点总出现次数。该索引用于路径存在性判断，不直接提供对应内容区的连续偏移。

文档级 Bloom 元数据。编码器将文档中出现的 token ID 加入 Bloom Filter。内容查询可先检查过滤器；确定不存在时跳过精确内容扫描，可能存在时仍须遍历内容流。Bloom 元数据与文档值一起封装，减少维护独立外部摘要的一致性问题，但会增加每个文档的表示空间。

长 token 内联区。对超过短 token 阈值的文本，内联区保存哈希标识、字节长度和原始解析后文本字节。内容流保存对应哈希标识。查询先以哈希匹配候选，再由精确字符串比较确认；该区不是全局字典的一部分，也不能把哈希值本身视作精确 token 身份。

节点结构流。前序结构流将路径标识和子节点数写入压缩值，供解码过程恢复层次和顺序。结构流不参与 Bloom 判断，也不直接作为 GIN 内容键。路径索引减少单文档路径判断工作；Bloom 过滤减少某些单文档内容扫描；GIN 在跨文档层面定位 token 候选；结构流支持解码。它们围绕同一中间表示协作，最终收益由整体存储和查询实验决定。

4.5 压缩表示的解码与正确性约束

表示正确性首先要求编码和解码采用相同的版本、区段边界和字典映射。解码器应验证总长度、偏移顺序和记录数量，逐项读取路径和值标识，并利用节点结构序列将叶内容放回相应节点。对短 token，通过字典反查获得文本；对长 token，通过内联区查找哈希标识并读取原文。

对符合原型输入约束的文档，若结构与内容编码使用一致的路径字典，短值字典在声明作用域内稳定，长值哈希候选始终经内联原文复核，且序列化长度和区段边界有效，则可逐项说明所定义操作的正确性：路径存在查询在包含所有节点路径的排序索引中查找目标路径 ID；精确内容查询先以 Bloom 排除确定不存在的标识，再扫描内容记录并比较完整 token；路径取值查询按文档序扫描与目标路径 ID 相同的首条记录；GIN 仅根据 token ID 形成候选，并由操作符复核处理哈希碰撞；解码则按前序结构流消费叶内容记录，恢复受支持语义中的节点边界、顺序和文本。该论证依赖前述不变量，并非对任意 XML 及任意损坏输入的形式化证明。

在本文原型声明的输入条件下，期望保持以下不变量：内容序列中的叶记录顺序与解析后文档中的叶节点顺序一致；每个内容记录中的路径 ID 与 token 标识均能映射回编码时的路径和值；路径索引的 pair_count 与内容流中相应路径 ID 的记录数一致；Bloom 若返回“不存在”，目标 token 不在构造过滤器时的 token 集合中，阳性只表示候选；结构流中的节点与内容记录按约定顺序消费，最终恢复元素顺序、边界和文本。

正确性比较包括元素顺序、层次边界和解析后的叶文本；不要求原始空白、实体拼写、属性顺序或字节序列完全一致。该性质由实现测试支持，有限测试不能替代对所有输入形式的证明。当前原型以 libxml 解析 UTF-8 文档，最大深度为 128，并拒绝属性、命名空间、文档类型定义（Document Type Definition, DTD）、注释、字符数据（CDATA）、处理指令、混合内容及部分文档级结构；该事实界定当前验证样本，不改变本文方法研究面向数据库 XML 压缩和索引复用的问题定位。

4.6　本章小结

本章提出面向查询的压缩表示方法：解析阶段生成路径—文本记录和节点结构记录；编码阶段确定并复用路径和值 ID；再将这些 ID 组织为文档序内容、排序路径摘要、Bloom 元数据、长值内联区和结构流。方法的关键是这些中间标识能够跨压缩、查询和索引阶段保持一致语义；其代价包括辅助信息、字典空间及长值处理。下一章说明如何基于该表示执行查询，并保证候选验证的正确性。

第五章　基于压缩表示的查询处理方法

5.1　查询模型与处理流程

设文档集合为 D，编码方法为 C。本文定义三类操作：

路径存在 EXISTS(D, p)

判断文档中是否存在完整路径 p，返回布尔值。

精确内容 CONTAINS(D, t)

判断是否存在叶文本完整值与 t 精确相等，返回布尔值；空字符串返回 false。

路径取值 GET(D, p)

按文档顺序返回路径 p 的首个叶文本；没有匹配路径时返回 NULL。

方法要求压缩表示上的查询结果与对解析语义执行相同操作的结果一致：

$$Q(C(D), x) = Q_{\text{ref}}(D, x), \qquad Q \in \{\text{EXISTS}, \text{CONTAINS}, \text{GET}\}.$$

该要求不是完整 XPath 等价。查询处理先将查询字符串映射为路径或 token 标识，再按操作类型访问排序路径索引、Bloom 元数据、内容流或 GIN，最后在可能存在哈希碰撞或近似命中时执行精确验证。各阶段复用编码时定义的标识规则，避免查询侧另造不一致的编码。

5.2　精确内容查询

对单文档 CONTAINS(D, t)，方法按 token 长度选择查询标识。短 token 通过共享字典映射为 token_id；若字典未命中，则该值不可能出现在采用同一字典的已编码短 token 记录中，可返回 false。长 token 计算内联哈希标识，但哈希只用于候选匹配，不能直接确认相等。

短 token 的文档内判断过程为：读取 token ID 并检查文档级 Bloom Filter；若 Bloom 明确返回不存在，立即返回 false；若 Bloom 返回可能存在，则顺序解码内容流中的 token ID 并比较。只有内容记录中出现目标 ID 才返回 true。长 token 查询先以哈希缩小待核验范围，再查找内联区原文并逐字节比较。即使不同文档中的不同长 token 发生 32 位

哈希碰撞，该检查也能避免错误命中；代价是候选文档仍需进入 recheck。

设文档包含 N 个内容对，Bloom 使用 k 次探测，则一次短 token 判断的工作量取决于字典查找、Bloom 探测和是否进行内容扫描。Bloom 阴性路径可避免行内扫描，但阳性路径最坏仍须扫描内容流；因此不能将所有负例统一描述为严格 $O(1)$。对于无索引全表查询，数据库执行器仍需对扫描行调用谓词，即使查询 token 未进入共享字典，也不能据此称整表处理只需一次字典查找。

5.2.1　示例文档上的正例与负例

以第 4 章示例文档为例，短文本 机械键盘的标识为 token_id=3。执行内容查询 CONTAINS(D, “机械键盘”) 时，查询字符串先映射为 3；若 Bloom Filter 返回阴性，则可以直接判定为 false；若返回阳性，则继续扫描内容流，在记录 $(6, 3)$ 处完成 token 标识匹配并返回 true。这里的 true 仍然来自内容记录的精确匹配，而不是 Bloom Filter 单独给出的结论。

查询 CONTAINS(D, “耳机”) 是一个负例。若 耳机未出现在共享 token 字典中，查询可以直接返回 false；若查询值已经进入字典但当前文档的 Bloom Filter 返回阴性，同样可以返回 false；若 Bloom 返回阳性，则仍需扫描全部候选内容记录，扫描结束后未发现目标 token 才能返回 false。由此可见，负例可能经过“字典未命中”“Bloom 确定不存在”或“精确扫描未匹配”三条路径，不能把所有负例都描述为相同的常数时间过程。

表 5.1　示例 XML 上的内容查询正例与负例

查询值	期望结果	主要处理过程
机械键盘	true	字典得到 token ID 3；Bloom 通过后扫描内容流并精确匹配 $(6, 3)$。
耳机	false	字典未命中，或 Bloom 确定不存在，或扫描内容流后未发现精确匹配。
1001	true	字典得到 token ID 1，在内容流首条记录 $(3, 1)$ 中匹配。
10	false	即使文本中包含字符 “10”，也不满足本文定义的完整叶文本相等语义。

表 5.1 中的最后一个负例用于说明本文的查询语义：CONTAINS 判断叶节点完整文本值是否与查询字符串逐字节相等，不执行任意子串匹配。该语义使压缩表示、GIN 候选键和精确 recheck 可以采用同一判断标准。

5.3 路径存在性与值提取查询

对 EXISTS(D, p)，查询字符串先在路径字典中查找 path_id。若路径未进入字典，则文档不可能包含该路径，可返回 false；若命中，则在该文档按 path_id 升序排列的路径索引中二分查找。设文档不同路径数为 P，路径索引部分需要 $O(\log P)$ 次比较，不包括字典查找及数据库函数调用成本。

对 GET(D, p)，方法同样先映射路径 ID 并检查路径索引。若不存在则返回 NULL；若存在，则从文档序内容流开头扫描，找到首个 path_id 相同的记录，再将 token ID 反查为字符串。由于相同路径记录可能在内容流中不连续，当前格式没有可直接定位到全部匹配值的偏移量，因此值提取部分最坏需要扫描 N 个内容对。该设计用保留文档序支持首值语义和结构恢复，代价是路径值提取仍需遍历。

路径索引还可能包含 pair_count $= 0$ 的内部节点路径：此时 EXISTS 返回 true，因为元素路径确实存在；GET 则在内容流中找不到对应叶文本并返回 NULL。这一区分来自两个操作各自的语义，不应把 pair_count $= 0$ 误认为路径不存在。路径索引只解决路径是否出现，不应写成 GET 的常数时间索引。若未来增加路径到内容偏移的辅助结构，还需处理同路径值不连续、重复顺序和元数据增量，并通过空间与查询实验评价净效果。

5.3.1 示例文档上的路径查询

在第 4 章示例文档中，路径 /orders/order/item/name 对应 path_id=6，因此 EXISTS(D, “/orders/order/item/name”) 先在路径字典中得到 6，再在排序路径索引中找到该条目，返回 true。相反，EXISTS(D, “/orders/order/price”) 因路径字典未命中而返回 false，不需要扫描内容流。

对于 GET(D, “/orders/order/item/name”)，路径索引只能确认路径存在，实际值仍需按文档序扫描内容流。内容流中的第一条匹配记录是 $(6, 3)$，通过 token 字典反查得到 机械键盘，因此函数返回该值；第二条 $(6, 5)$ 对应 鼠标，在当前“返回首个匹配值”的语义下不会被返回。该例说明，路径索引解决的是路径存在性，而文档序内容流承担首值定位和重复路径的顺序语义。

表 5.2　示例 XML 上的路径查询正例与负例

查询	期望结果	说明
EXISTS(/orders/order/item/name)	true	路径索引存在 path_id=6。
EXISTS(/orders/order/price)	false	路径字典未命中，不访问内容流。
GET(/orders/order/item/name)	机械键盘	扫描到首条 (6,3) 后返回 token 的反查值。
GET(/orders/order/discount)	NULL	路径不存在，无法找到匹配内容记录。

5.4　路径索引、Bloom 过滤与候选验证

路径索引、Bloom 和 GIN 面向不同范围。路径索引在单文档内定位路径标识；Bloom 在单文档内快速否定 token；GIN 在整个表范围内为内容 token 生成候选行。其协作关系是：GIN 先减少需要访问的行，逐行谓词及必要的精确复核保证操作语义；没有 GIN 时，Bloom 可在顺序扫描的每个文档内减少部分内容扫描。GIN 不是利用每个文档的 Bloom 位图构造倒排项。

Bloom 的正确性边界是“阴性可排除，阳性需确认”。GIN 的正确性边界是“索引键匹配生成候选，近似键需要 recheck”。二者不可混为一谈。Bloom 假阳性会增加内容扫描但不改变结果；长 token 哈希碰撞会增加索引候选，但必须通过实际字符串比较排除错误命中。

当前 GIN 的 extractValue 从内容流解码 token ID 并提取不同键，索引构建阶段因此仍有扫描和键提取工作；它不必从原始 XML 重新解析或执行通用文本分词。查询侧 extractQuery 使用与编码相同的短 token 字典或长 token 哈希规则。consistent 将可能命中的候选标记为需要 recheck；三值一致性路径将长 token 哈希候选保留为 MAYBE，避免错误地因索引键相同而跳过堆上精确检查。

5.5　PostgreSQL 操作符及 GIN 索引实现

本文原型将方法接入 PostgreSQL 自定义类型。输入函数将 XML 外部文本转换为压缩表示；输出函数根据结构流解码为 XML 文本表示。查询函数分别实现 xml_contains、xml_exists 和 xml_get。其中内容包含操作注册为可由 GIN 使用的操作符，GIN 操作类提供值键抽取、查询键映射和一致性判断回调。

对于短 token，extractValue 从压缩内容流提取 token ID，extractQuery 通过共享字典得到查询 ID。字典未命中时，查询回调返回零个键，GIN 可直接得到空候选集。对于长 token，查询键使用内联哈希 ID，仍要求候选精确复核。索引端只负责高效生成候选，不改变 SQL 操作符定义的精确 token 包含语义。

PostgreSQL 在本文中是方法的实现与验证平台。压缩编码中间数据复用原则理论上可以映射到其他存储引擎的自定义类型或索引接口，但各平台的字典持久化、多版本并发控制（Multi-Version Concurrency Control, MVCC）、应用程序编程接口（Application Programming Interface, API）、崩溃恢复和代价模型均不同。当前没有其他数据库上的实现或实验，因此这里只说明可迁移研究方向，不把迁移后的行为写成已验证结论。

5.6 本章小结

本章说明基于压缩表示的查询处理方法。精确内容查询利用字典、Bloom 和内容流分层判断；路径存在查询通过字典和排序路径索引查找；路径取值查询在路径确认后扫描文档序内容；GIN 负责跨文档候选定位，最终结果由精确谓词复核。方法复用的是编码阶段的语义 ID，Bloom、路径索引和 GIN 则在不同范围内发挥作用。下一章通过实验检验这些过程的正确性与成本。

第六章　实验评估与结果讨论

6.1　研究问题与评价指标

实验围绕前三章提出的方法问题展开，而不是把实验设计成与方法并列的研究主题。本文考察以下问题：

RQ1：正确性

压缩与解码是否保留当前输入范围内定义的结构语义，查询结果是否与独立预言机一致？

RQ2：空间

编码中间信息复用后的列值、完整 relation、索引和共享字典分别占用多少空间？

RQ3：查询

直接查询压缩表示及使用 GIN 时，在当前查询语义和环境下观察到怎样的延迟？

RQ4：代价

Bloom 与路径索引分别带来怎样的性能变化？现有写入测量能否评价本次测试的压缩表示？

相应指标包括结构差分与查询结果一致性、平均列值大小、表及 relation 大小、GIN 与共享字典字节数、查询延迟（实例中位数）以及开关 Bloom 或路径索引后的描述性差异。列值压缩率、relation 总量和查询延迟分别回答不同问题，不合并成单一“整体提升”指标。现有写入吞吐测量的表示与本次查询、存储实验不一致，故不作为本方法配置的写入评价指标。

本实验定位为原型的可行性验证和基本性能观测，不以全面复现所有 XML 压缩论文为目标，也不声称覆盖所有生产负载。

6.2　实验环境、数据集与对比方案

查询与存储实验运行于 Linux 7.3.0-9-generic x86_64、glibc 2.44，使用启用 libxml 的 PostgreSQL 16.11（x86_64-pc-linux-gnu，由 GCC 15.3.0 编译）；本文扩展、PostgreSQL 和比较器均以 O2 构建。三次独立运行采用 fsync=on、full_page_writes=on、synchronous_commit=on、jit=off、max_parallel_workers_per_gather=0、default_toast_compression=

及 shared_buffers=16384。CPU 型号、物理内存和 zlib/zstd 运行库版本未记录。

输入为按文件名排序的导航 XML 文档前缀。10,000 文档数据集的 SHA-256 为 124e6287a1826a20918b2c07d54b1c469c578b029f8bc34805d3457825fa57b0。1K、5K 和 10K 规模分别使用独立新建的 PostgreSQL 数据目录；10K 完成三次独立重复。每个查询先预热 5 次，再正式测量 10 次；报告跨实例中位数的中位数。该协议对应 warm-cache 测量，不是冷盘延迟。

查询对照包括本文压缩表示的顺序 exact-token 查询及 GIN 查询、保留原始 XML 文本并生成短叶 token 数组后建立 GIN 的工程基线、原生 XML 上语义对应的 exact-token XPath 查询，以及 zlib-9、zstd-3 和 zstd-19 压缩后逐行解压并解析 XML 的无索引扫描基线。通用压缩基线用于观测纯压缩表示上的查询处理代价，不作为索引系统对照。路径存在及路径取值另与相应消融或参照操作核验。测试 token neiye_exto_ver 命中 1,816 行；缺失 token 命中 0 行。

空间对照须区分配置：原生 xml relation 是无索引存储参考；本文压缩表示的 relation 包括表和 GIN；text[] + GIN 基线保留原始 XML 文本，并额外物化 token 数组及其索引。该基线实现本实验的精确短 token 包含语义，不是 PostgreSQL 在原生 xml datum 上直接建立 GIN。实验记录未包含 CPU、物理内存和部分压缩库版本，故不以当前机器信息代替缺失的实验机信息。

6.3 编码正确性与存储开销分析

正确性结果。全量结构差分覆盖 10,000 个 XML 输入/输出；解码后元素顺序、容器边界和叶文本差异均为 0。每个 benchmark 实例的 44 个查询条件均与独立 Python 预言机一致。PostgreSQL 16.11 隔离实例的 14 项回归检查通过，覆盖结构语义、GIN 长 token 碰撞候选、持久化、重启和路径查询。以上结果支持已测样本与测试条件下的实现正确性，不等于对所有 XML 输入进行形式化证明。

列值空间。 10,000 行时本文压缩表示的平均列值为 1,818.63 B，原生 XML 列为 2,278.21 B，列级观测减少 20.17%。此结果说明当前数据和编码配置下列值较小，不代表完整 relation 或系统总存储一定较小。

为区分压缩字节数与可查询压缩表示，实验还测量了将原始 XML 分别以 zlib-9、zstd-3、zstd-19 压缩为 bytea 的无索引基线。10K 时三者平均列值分别为 1,637.30 B、

1,645.94 B、1,524.99 B，均小于本文压缩表示的 1,818.63 B；对应无索引 relation 大小分别为 18,432,000 B、18,333,696 B、17,104,896 B。压缩基线 relation 不含查询索引，本文方法 relation 则含 GIN，因此二者完整 relation 不能作对称系统比较。压缩基线支持本实验的精确叶文本查询，但查询时需对每行解压并解析 XML，不具备本文方法的路径/内容 ID 及 GIN 键提取机制；其结果用于展示纯压缩空间参照和顺序查询代价，不用于声称本文方法压缩率优于通用压缩器。

表 6.1　10K 行数据的无索引压缩基线

算法	平均列值（B）	relation（B）	测试查询处理
zlib-9	1,637.30	18,432,000	顺序扫描、逐行解压并解析 XML
zstd-3	1,645.94	18,333,696	顺序扫描、逐行解压并解析 XML
zstd-19	1,524.99	17,104,896	顺序扫描、逐行解压并解析 XML

完整空间对比见表 6.2。第一行无索引，第二、三行含索引，因此原生 XML 的 25,640,960 B 只能作为不对称配置的存储参考，不能据此判断本文含 GIN 的 relation 比含同类索引的原生 XML relation 大或小。与同测试语义的 text[] + GIN 基线相比，本文方法完整 relation 小 50.49%。共享字典文件另有 9,771,540 B；将一份字典全额计入后，测量持久化数据为 39,803,412 B，较该基线 relation 小 34.38%。该核算不含数据库级及文件系统级其他开销，也未将字典在多表之间摊销；TOAST 已包含在 relation 统计中，不重复加算。

表 6.2　不同存储与查询方案的空间占用对比

存储和查询方案	表大小（B）	GIN 大小（B）	完整 relation（B）	口径说明
原生 PostgreSQL XML，无索引	25,640,960	0	25,640,960	原生 XML 存储参考
本文压缩表示 + GIN	20,987,904	9,043,968	30,031,872	压缩值及 token GIN
原始 XML text + 生成的 text[] + GIN	35,659,776	24,993,792	60,653,568	相同 exact-token 语义的工程索引基线

6.4　查询性能与索引效果分析

10K warm-cache 三次独立实例的测量结果见表 6.3。首先比较精确叶文本 token 查询：zlib/zstd 基线通过自定义函数逐行解压并解析 XML 后精确比对叶文本，均为无索

引顺序扫描。其查询语义与本文方法的 exact-token 查询一致，但处理路径和索引配置不同，结果应理解为当前实现条件下的观测。

表 6.3　10K warm-cache 查询延迟比较

查询条件	r_1（ms）	r_2（ms）	r_3（ms）	跨实例中位数的中位数（ms）
本文方法顺序扫描，Bloom 开	72.984	87.450	73.102	73.102
本文方法顺序扫描，Bloom 关	85.433	77.111	100.729	85.433
本文方法 + GIN	28.246	27.032	24.934	27.032
text[] + GIN	27.020	29.911	24.713	27.020
zlib-9，无索引解压解析扫描	1,808.592	1,395.810	1,421.261	1,421.261
zstd-3，无索引解压解析扫描	1,455.782	1,329.179	1,429.047	1,429.047
zstd-19，无索引解压解析扫描	1,860.057	1,455.615	1,468.413	1,468.413
原生 XML exact-token xpath_exists	2,220.014	1,763.064	1,650.346	1,763.064

本文方法与 text[] + GIN 的中位延迟分别为 27.032 ms 和 27.020 ms，几乎相同。该结果支持“压缩值可以为 GIN 提供可用索引键，且本测试延迟与该工程基线接近”，不支持本文方法 GIN 查询更快的结论。与原生 XML exact-token XPath 的差异是本轮特定表达式、数据集和构建下的观测，不应扩展为所有 XPath 查询的普遍加速比。

通用压缩基线虽然平均列值更小，但其解压加 XML 解析的逐行扫描在本次观测中的中位数约为 1.42–1.47 s；本文方法顺序扫描中位数为 73.102 ms。该差异与编码复用减少查询阶段解压/解析工作的机制相一致，但由于原型、解析实现和压缩库执行路径不同，只能作为方法机制的初步性能观测，不能据此推导跨实现、跨数据集的普遍加速结论。

Bloom 开关下顺序查询观察到 73.102 ms 与 85.433 ms；路径索引开关下路径查询观察到 104.371 ms 与 109.618 ms。缺失 token 的 GIN 查询三轮中位数为 0.075、0.075 和 0.074 ms，表明该测试条件下字典未命中路径可快速返回；不能将其推广为所有负例或规模下的严格常数时间。路径索引开启时原始值为 104.371、107.629、104.064 ms，关闭时为 121.733、109.618、96.323 ms；对应实例中位数的中位数分别为 104.371 ms 和 109.618 ms。实验仅有三次独立实例且未进行统计显著性检验，因此这些结果仅作为描述性消融。

6.5　写入开销与消融实验

本文方法的写入过程包括 XML 解析、路径和值标识分配、共享字典访问、压缩序列化及索引维护，因而需要独立评价写入代价。目前没有与本章存储和查询实验使用同一二进制表示的写入吞吐测量。已有早期写入记录对应另一种表示，不能与本章结果合并，也不能用于判断本文所测表示相对原生 XML 或文本存储的写入性能。解析、字典访问、压缩编码与预写式日志（Write-Ahead Logging, WAL）的分项成本及并发吞吐仍待测量。

消融结果已在本章查询分析中分别报告 Bloom 和路径索引开关。在同一数据集上关闭 Bloom 后，顺序 exact-token 查询中位数从 73.102 ms 变为 85.433 ms；关闭路径索引后，路径查询中位数从 104.371 ms 变为 109.618 ms。由于重复次数有限且未做显著性检验，这些值只能说明本轮运行中的方向和量级，不能证明其他文档分布下有相同收益。共享字典是当前二进制标识契约的基础，现有实验未提供“取消共享字典但保持其他语义不变”的可比消融，因此不对该项作单独收益归因。

6.6　结果分析与有效性威胁

综合 RQ1–RQ4，当前实验支持以下条件化判断：编码阶段形成的路径和值标识能够在当前原型内被压缩表示、指定查询和 GIN 键提取共同消费；结构流、Bloom Filter 和精确 recheck 为解码及候选验证提供了约束；在所测数据和配置下，压缩列值较原生 XML 列值减少，GIN 查询延迟与 text[] + GIN 工程基线接近。上述判断描述的是方法链的可实现性及本轮空间、查询权衡，不是对所有 XML 工作负载的性能排名。

本章结果存在以下有效性威胁。第一，数据集为单一导航 XML 文档前缀，尚未覆盖不同路径深度、值分布、重复率和长 token 比例；因此空间收益不能直接推广到其他 XML 数据。第二，查询测量采用 warm-cache 协议，且实验记录缺少 CPU、物理内存及部分压缩库版本，冷盘延迟和跨机器可复现性仍需补充。第三，原生 XML、无索引通用压缩基线和含 GIN 的本文方法配置并不对称，空间结果必须按列值、relation、索引和字典分别解释。第四，当前写入测量与本章压缩表示不一致，不能据此断言本文方法的写入吞吐或更新代价。第五，原型仅实现本文定义的 XPath 导向查询操作，完整 XPath 语法、命名空间、属性轴及更多 XML 节点类型尚未在本实验中覆盖。

因此，本文支持的是“编码阶段形成的中间标识能够在当前原型内被压缩表示、指定查询和 GIN 键提取复用，并在所测条件下保持正确”的结论；不能单独证明该方法对所有数据库、任意 XML 输入或任意查询负载都更优。

6.7 本章小结

本章按照正确性、空间、查询和写入成本评价本文方法。正确性检查在已测文档和查询条件上通过；本文压缩列值小于原生 XML，但大于无索引 zlib/zstd 压缩基线，后者 relation 不含索引，不能与本文含 GIN 的配置作对称比较。与同语义 text[] + GIN 基线相比，GIN 延迟基本相当，计入一份字典后的测量持久化成本仍较低。当前尚无与本章表示匹配的写入吞吐证据。所有定量结论均限制于对应数据、环境和测试协议。

结　论

本文围绕面向数据库 XML 类型的压缩与索引优化方法，研究编码阶段产生的路径、文本和值标识如何继续服务于查询处理和索引构建。在梳理 XML 压缩、压缩域查询以及路径和内容索引研究的基础上，本文提出以中间标识复用为核心的压缩表示方法，给出精确内容、路径存在和路径取值的查询处理过程，并通过 PostgreSQL 原型进行正确性与基本性能验证。

本文表示将共享字典 ID、Varint 内容流、排序路径索引、Bloom 元数据、长值内联区和前序结构流纳入统一数据契约。查询方法在这些结构上分别完成候选生成、否定过滤和精确确认，并使 GIN 操作类从压缩值提取与查询同语义的 token 键。原型当前实现的解析与查询能力界定了实验验证范围，而不是本文方法的研究主题。主要研究结论如下。

第一，结构感知压缩与压缩编码复用是本文方法的核心。本文不把 XML 仅视为一段文本进行通用压缩，而是识别其层次、路径和内容特征，对重复出现的路径与内容进行字典编码，减少原始标签和字符串的重复存储。编码过程中形成的路径 ID 和 token ID 同时服务于压缩表示、查询处理与 GIN 键抽取，使压缩产生的中间数据得到复用。必要的结构信息由结构流保留，以支持解码和结构恢复；索引返回的候选结果则通过精确 recheck 验证。结构差分、查询预言机和回归用例表明，这条方法链在已测范围内能够正确实现。

第二，本文追求的不是单列空间最小，而是在支持高效查询的前提下减少存储开销。与以压缩率为主要目标的 zlib、zstd 相比，结构感知表示需要保留供查询使用的编码和辅助信息，因此单列压缩率并非最优；但在本次 10K 数据实验中，其列值仍比原生 XML 小 20.17%。与支持相同 exact-token 查询的 $\mathrm{text}[] + \mathrm{GIN}$ 工程基线相比，本文方法的查询延迟基本相当；将一份共享字典全额计入后，测量持久化成本低 34.38%。这些结论仅适用于本次数据、实现和统计口径。

在一个简化模型中，设重复结构出现 N 次，不同路径数为 P；原始表示中每次重复结构平均占 s 字节，编码后每次平均占 e 字节；字典及路径元数据等固定或随路径数

变化的开销记为 $H(P)$。则可近似表示为

$$\frac{S_{\text{enc}}}{S_{\text{raw}}} \approx \frac{Ne + H(P)}{Ns} = \frac{e}{s} + \frac{H(P)}{Ns}.$$

该关系说明：当数据增长主要来自相似结构重复、P 相对 N 增长较慢，且 s、e 与元数据函数 $H(P)$ 的假设大体稳定时，元数据摊销项 $H(P)/(Ns)$ 会随 N 增大而减小，压缩比趋向由 e/s 决定。数据越密集、结构越相似，若它们具体体现为重复结构数量增加而不同路径种类没有同比增加，则同样会摊薄路径字典及辅助结构的单位开销。该式是基于简化假设的趋势分析，不是对任意 XML 数据分布的普遍定理；实际效果仍受文本值分布、长 token 比例、结构流和 Bloom 参数影响。

第三，压缩表示能够支持查询，而不必先还原完整 XML。在本文相同 exact-token 查询语义下，本文方法与 $\text{text}[] + \text{GIN}$ 工程基线延迟基本持平（27.032 ms 与 27.020 ms）。编码阶段形成的 token ID 可用于 GIN 键抽取和查询匹配；对单个文档，Bloom Filter 能在多数阴性场景快速判定 token 不存在，可能存在的结果仍需经过内容流精确检查。因此，本文的查询价值不在于证明比该基线更快，而是在获得一定空间节省的同时，保持与本次测量中高效索引方案相当的查询性能。

参考文献

[1] World Wide Web Consortium. Extensible Markup Language (XML) 1.0 (Fifth Edition). W3C Recommendation, 2008. https://www.w3.org/TR/xml/

[2] PostgreSQL Global Development Group. XML Type. PostgreSQL 16 Documentation, Section 8.13, accessed 2026-10-25. https://www.postgresql.org/docs/16/datatype-xml.html

[3] PostgreSQL Global Development Group. XML Functions. PostgreSQL 16 Documentation, Section 9.15, accessed 2026-10-19. https://www.postgresql.org/docs/16/functions-xml.html

[4] 王腾蛟, 高军, 杨冬青, 唐世渭, 刘云峰. 面向 XPath 执行的 XML 数据流压缩方法 [J]. 软件学报, 2005, 16(5): 869–877. DOI: https://doi.org/10.1360/jos160869

[5] Liefke, H., and Suciu, D.“XMill: An Efficient Compressor for XML Data.”Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, pp. 153–164, 2000. https://doi.org/10.1145/342009.335405

[6] Tolani, P. M., and Haritsa, J. R. “XGrind: A Query-Friendly XML Compressor.”Proceedings of the 18th IEEE International Conference on Data Engineering, pp. 225–234, 2002. https://dsl.cds.iisc.ac.in/publications/conference/xgrind.pdf

[7] Min, J.-K., Park, M.-J., and Chung, C.-W.“XPRESS: A Queriable Compression for XML Data.”Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data, pp. 122–133, 2003. https://doi.org/10.1145/872757.872775

[8] Buneman, P., Grohe, M., and Koch, C.“Path Queries on Compressed XML.”Proceedings of the 29th International Conference on Very Large Data Bases, pp. 141–152, 2003. https://doi.org/10.1016/B978-012722442-8/50021-5

[9] Arion, A., Bonifati, A., Manolescu, I., and Pugliese, A. “XQueC: A Query-Conscious Compressed XML Database.”ACM Transactions on Internet Technology, 7(2), Article 10, 2007. https://doi.org/10.1145/1239971.1239974

[10] Müldner, T., Fry, C., Miziołek, J. K., and Durno, S. “XSAQCT: XML Queryable Compressor.” Balisage Series on Markup Technologies, 3, 2009. https://doi.org/10.4242/BalisageVol3.Muldner01

[11] Cheng, J., and Ng, W. “XQzip: Querying Compressed XML Using Structural Indexing.” Proceedings of the 8th International Conference on Extending Database Technology, Lecture Notes in Computer Science 2992, pp. 219–236, 2004. https://doi.org/10.1007/978-3-540-24741-8_14

[12] Ferragina, P., Luccio, F., Manzini, G., and Muthukrishnan, S. “Compressing and Searching XML Data Via Two Zips.” Proceedings of the 15th International Conference on World Wide Web, pp. 751–760, 2006. https://doi.org/10.1145/1135777.1135891

[13] Brisaboa, N. R., Cerdeira-Pena, A., and Navarro, G. “XXS: Efficient XPath Evaluation on Compressed XML Documents.” ACM Transactions on Information Systems, 32(3), Article 13, 2014. https://doi.org/10.1145/2629554

[14] Arroyuelo, D., Claude, F., Maneth, S., Mäkinen, V., Navarro, G., Nguyễn, K., Sirén, J., and Välimäki, N. “Fast In-Memory XPath Search Using Compressed Indexes.” Software: Practice and Experience, 45(3), pp. 399–434, 2015. https://doi.org/10.1002/spe.2227

[15] Goldman, R., and Widom, J. “DataGuides: Enabling Query Formulation and Optimization in Semistructured Databases.” Proceedings of the 23rd International Conference on Very Large Data Bases, pp. 436–445, 1997. https://www.vldb.org/conf/1997/P436.PDF

[16] Cooper, B. F., Sample, N., Franklin, M. J., Hjaltason, G. R., and Shadmon, M. “A Fast Index for Semistructured Data.” Proceedings of the 27th International Conference on Very Large Data Bases, pp. 341–350, 2001. https://www.vldb.org/conf/2001/P341.pdf

[17] Harding, P. J., Li, Q., and Moon, B. “XISS/R: XML Indexing and Storage System Using RDBMS.” Proceedings of the 29th International Conference on Very Large Data Bases, pp. 1073–1076, 2003. https://www.vldb.org/archives/website/2003/papers/S35P06.pdf

[18] Pospíšil, T., and Chmelař, P. “FastX – XML Indexing for PostgreSQL.” Software/project record, Faculty of Information Technology, Brno University of Technology, 2011. https:

//www.fit.vut.cz/research/result/c90534/.cs

[19] PostgreSQL Global Development Group. GIN Indexes and GIN Extensibility. PostgreSQL 16 Documentation, accessed 2026-10-19. https://www.postgresql.org/docs/16/gin.html; https://www.postgresql.org/docs/16/gin-extensibility.html

[20] PostgreSQL Global Development Group. TOAST (The Oversized-Attribute Storage Technique). PostgreSQL 16 Documentation, Section 73.2. https://www.postgresql.org/docs/16/storage-toast.html

[21] Williams, H. E., and Zobel, J. “Compressing Integers for Fast File Access.” The Computer Journal, 42(3), pp. 193–201, 1999. https://doi.org/10.1093/comjnl/42.3.193

[22] Bloom, B. H. “Space/Time Trade-Offs in Hash Coding with Allowable Errors.” Communications of the ACM, 13(7), pp. 422–426, 1970. https://doi.org/10.1145/362686.362692